CS-3083 Project 1 (300 pts)

In this project, you will use Python to implement A* search, simulated annealing and genetic algorithms.

You can only use the following Python modules in your programs:

- heapq
- numpy
- matplotlib.pyplot
- time
- os

Task 1: making a 3-by-3 magic square (100 pts)

In this task, your program should first randomly generate an initial 3-by-3 square containing numbers 1~9. A possible square might be:

1	3	9
2	6	4
8	7	5

Then, your program should use A* search and UCS to search for the path to turn the initial square into the following magic square:

8	**1**	**6**
3	**5**	**7**
4	**9**	**2**

(Note: there are 8 different magic squares. To simplify your task, we set this one as the goal state)

The only allowed actions to change one state to a different one are swapping between two adjacent tiles. For example, in the first given square, from the viewpoint of tile 3, it can only swap with tile 1, tile 6 or tile 9. An action of swapping between 3 and 9 results in a new state:

1	9	3
2	6	4
8	7	5

As A* search is an informed search, your program **must design at least one heuristic** to evaluate the cost of turning any state into the goal state**. At least one of your heuristic functions** must be admissible, which will guarantee that the solution found by using it is cost-optimal.

Evaluation Requirements

Based on the same initial state, compare your A* search with UCS in terms of the number of expanded nodes, total execution time, and solution's path-cost. Run both algorithms at least 10 times before you draw any conclusions.

Submission Requirements

1. Your python program (please name it "MS_YourLastName_YourFirstName.py")
2. A report covering the following contents:

(1) How you implemented A* search

(2) How you defined your heuristic function(s). Provide qualitative and/or quantitative proof that one of the heuristics is admissible

(3) Comparison between your A* search under different heuristic settings and the UCS. Tables and/or plots showing the number of expanded nodes, total execution time and solution's path-cost must be included. You also need to analyze the results and explain any interesting patterns you noticed.

Task 2: Traveling Salesperson Problem (TSP) (100 pts)

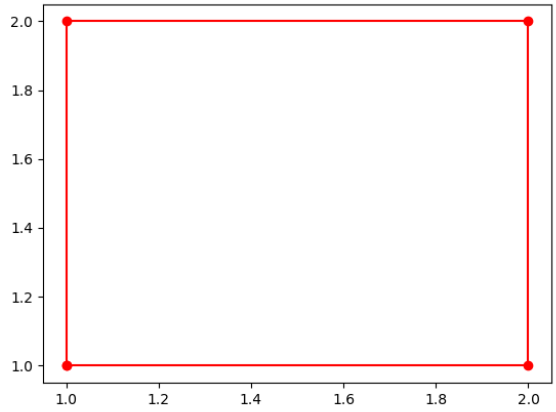
In this task, you will use the Simulated Annealing (SA) algorithm and a brute-force method to solve the TSP problems. You will evaluate these two methods in terms of optimality, execution time and stability.

Your program should read input from the provided text files (TSP1.txt~TSP4.txt). Each text file defines the input for one TSP problem, which consists of the coordinates of a set of cities. Assume that there are direct flights between every pair of cities, and each flight follows a straight-line path. Consider the cost of traveling from one city to another as the straight-line distance between the two cities. Your programs should target finding a tour of the cities with the minimum total cost.

For example, TSP1.txt provides a very simple case to help verify your program's correctness:

1 1
2 2
1 2
2 1

, where the 3rd line defines a city located at (x=1, y=2). Plotting the 4 cities in the coordinate system:



If your brute-force TSP solver works correctly, the output tour should connect the four cities as shown in the picture. For the SA TSP solver, you need to tune its parameters and settings to acquire the optimal solution. A well-tuned SA TSP solver will have a good chance to find the optimal solution.

Your SA TSP solver should start with a randomly generated tour, and possible actions on the current tour are the swapping of any two cities. Your brute-force TSP solver can fix the starting city on any one of the cities, for instance, the first city on the text file.

Evaluation Requirements

The SA TSP solver needs thorough tuning and tests to achieve the best performance. You should try different combinations of its control parameters. For a potentially good combination, run as many tests as you can to confirm its stability. Collect and analyze the results in your report.

Submission Requirements

1. Your Python program (Please name it TSP_YourLastName_YourFirstName.py)
2. A report covering the following contents:

(1) How you designed the SA TSP solver: what is the equation you used to convert loop counting into temperature? Did you try multiple equations? Which one worked the best?

(2) How you tuned the control parameters to achieve the best result for the SA TSP solver? Is the performance stable on each TSP across multiple runs? Are parameters identical for all TSPs or different for each TSP? Which parameters have the largest impact on the correctness and execution time?

(3) Use the 12-city input TSP4.txt to execute your best-tuned SA TSP solver one time and make 4 plots:

- iteration index (x-axis) vs current temperature (y-axis)
- iteration index (x-axis) vs current tour's cost (y-axis)

- iteration index (x-axis) vs random-walk occurrence (y-axis: 1 if random walk happens, 0 otherwise)
- The output tour like I did in the previous 4-city example.

Make comments for all the plots.

(4) How you implemented your brute-force TSP solver?

(5) Comparison between SA and brute-force TSP solvers: (run the SA solver multiple times before you draw any conclusions)

- Correctness (are the two solvers always correct?)
- Execution time (which solver is faster? Any input causes the brute-force solver to run for a very long time? What is the performance of the other solver on this input? Explain the observed difference)

(6) Based on the above results, what do you think of the two solvers and what are their pros and cons? What are the best scenarios to use each?

Task 3: 3-SAT problem (100 pts)

In this task, you will implement the genetic algorithm (GA) to solve the 3-SAT problem.

The input of your program is provided as a text file containing all the clauses of a CNF. Each line of the text represents one disjunctive clause. For example, the provided 3SAT_4_6.txt contains the following 6 lines:

x0_pos x1_pos x2_pos
x0_pos x1_neg x3_pos
x1_pos x2_pos x3_pos
x0_pos x2_pos x3_pos
x1_pos x2_neg x3_neg
x0_neg x1_neg x3_neg

, where the 2nd line represents the clause $x_0 \vee \overline{x_1} \vee x_3$. Note that _pos means the boolean variable is in its positive literal form, and _neg means the negative literal form. The 6 lines in the text together define a 4-variable and 6-clause 3-SAT problem:

$$\Phi = (x_0 \vee x_1 \vee x_2) \wedge (x_0 \vee \overline{x_1} \vee x_3) \wedge (x_1 \vee x_2 \vee x_3) \wedge (x_0 \vee x_2 \vee x_3) \wedge (x_1 \vee \overline{x_2} \vee \overline{x_3}) \wedge (\overline{x_0} \vee \overline{x_1} \vee \overline{x_3})$$

Your GA 3-SAT solver needs to assign each variable with a true or false value, and the target is to make Φ true. Note that the 3-SAT problems are NP-hard. This toy example might take a very short time to try all the $2^4 = 16$ combinations on the 4 variables x0~x3. However, for larger 3-SAT problems, for instance 100 variables, the number of combinations to test in a brute-force method would be 2^{100}, which will take forever to finish. This is where local search strategies come in and the GA you just learned in our lectures is one of them. Follow the textbook description of the GA (section 4.1.4 and Figure 4.8) to implement your program.

You are provided with 3SAT_4_6.txt, 3SAT_24_100.txt, 3SAT_100_100.txt and 3SAT_100_500.txt to test your program. The smallest input can help check your program's correctness (if it is lucky enough to find the solution), and the bigger ones can further test your program's scalability.

Also provided to you is a python program 3SAT_generator.py to generate random 3-SAT problems. You can change its n (number of variables) and m (number of clauses) to randomly generate differently sized 3-SAT problems. Please note that the generated 3-SAT will be written into a text file named "3SAT_yourNvalue_yourMvalue.txt", and any existing text with the same name will be overwritten.

Evaluation Requirements:

The GA algorithm has many control parameters to tune to achieve a better performance (a balance between running time and relative correctness). You should tune at least the following parameters and report the effects in your report: generation size (how many individuals in a generation), number of generations (number of iterations) and mutation rate. You should also try different ways of make crossover point(s) to recombine two selected individuals into a child individual.

Submission Requirements:

1. Your python program (please name it "3SAT_YourLastName_YourFirstName.py")
2. A report summarizing your implementation and findings:

(1) How you designed your GA program. I am especially interested in how you define and calculate the fitness score, how the score is turned into the probability of selection, how two selected individuals are recombined based on some calculated crossover point(s), how mutations are carried out each time, and how the next generation is finally made.

(2) Run your best tuned program on the 3SAT_100_100.txt input one time and make a single plot with three curves:

- Generation index vs max_fitness
- Generation index vs min_fitness
- Generation index vs average_fitness

Are these curves consistent with your expectations? Analyze your findings.

(3) A thorough discussion of your findings from tuning the control parameters of your GA program. Please include any necessary plots and analysis to help make your story.

(4) For input of different sizes (n and m values), is your GA 3SAT solver always able to find a satisfying assignment (making the CNF true)? As you fix the number of variables n=100 and tune the number of clauses m, regarding satisfiability, what ratios of m/n generally make the CNF unsatisfiable and what ratios make it satisfiable, any ratios in between? Record your findings here, and we will soon revisit this problem in a different chapter.

3. A summary about what you learned from the entire project 1. Keep in mind that the three tasks are all very basic but hard to solve with brute-force methods.